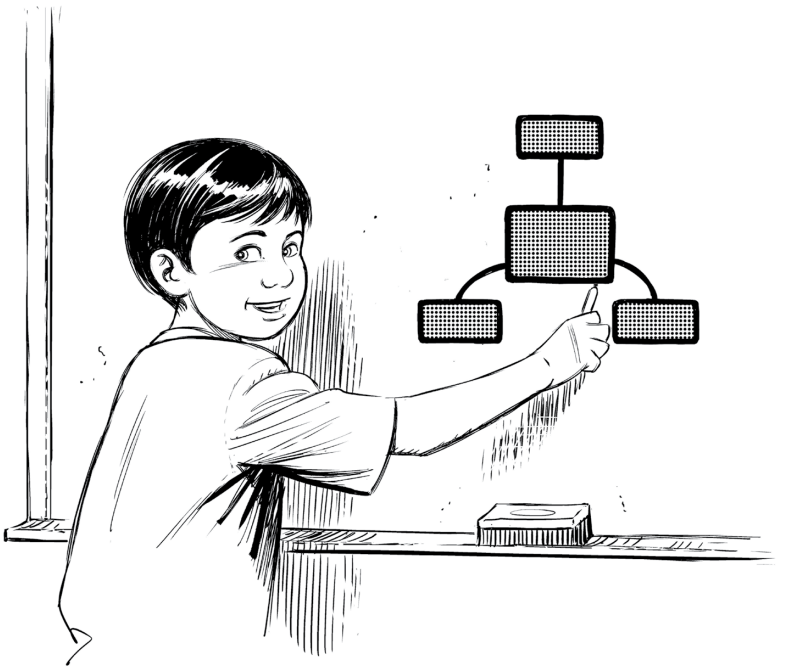
2 architectural characteristics
Know Your Capabilities



What does your architecture need to support? Architectural characteristics (the capabilities of an architecture) are the fundamental building blocks of any system. Without them, you cannot make architectural decisions, select an architectural style, or in many cases even create a logical architecture. In this chapter you'll learn how to define some of the more common characteristics (like scalability, reliability, and testability), how they influence a software architecture, how they help you make architectural decisions, and how to identify which ones are important for your particular situation. Ready to add some capabilities to your software architecture?

Causing Lafter

Sillycon Symposia is a startup with a Bay Area feel whose business plan combines technology-themed conferences with comedy. By gathering like minds, Sillycon provides unique offerings for each group and keeps them engaged by keeping them laughing.

How hard could it be to start a social networking site?!

Part of the business plan includes building Lafter, a social media network related to (but not limited to) the conferences Sillycon hosts. The business stakeholders put together a ***requirements document*** for it:

The good news is that they've got the logo figured out. The rest is up to you.



A pretty standard level of detail for a requirements document

Sillycon Symposia is hosting a social media network of like-minded technologists named Lafter.

Users: Hundreds of speakers, thousands of users

Requirements:

Users can register for usernames and approve the privacy policy

Users can add new content on Lafter as a "Joke" (long-form post) or "Pun" (short-form post)

Followers can "HaHa" (indicating strong approval) or "Giggle" (a milder approval message) content they like

Speakers at Sillycon Symposia events have a special icon

Speakers can host forums on the platform related to their content

Users can post messages of up to 281 characters

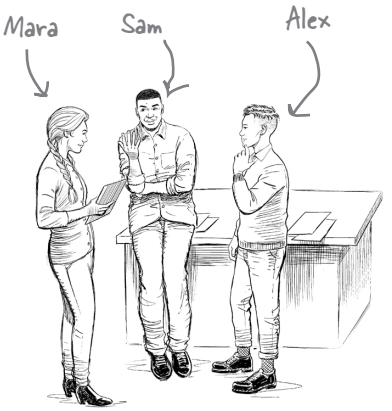
Users can also post links to external content

Additional Context:

International support

Very small support staff

"Bursty" traffic: extremely busy during live conferences

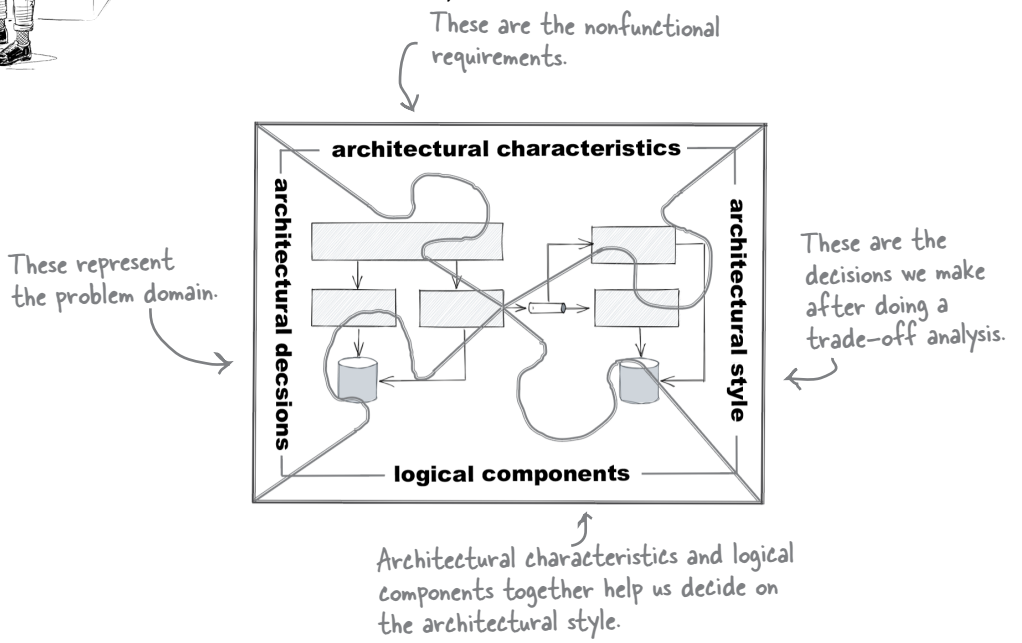


Cubicle conversation

Alex: Look what just landed in my inbox—the Powers That Be want me to be the architect for the Sillycon Symposia social media app, Lafter.

Sam: You have the requirements? You should jump right into the design of the system—it seems really straightforward.

Mara: Well, you can really only do that for the simplest of applications, and I don't think this one qualifies. Remember the diagram I drew on the whiteboard the other day?



You need to analyze both architectural characteristics and logical components before you can choose an architectural style as a starting point.

You can implement just about any application in any architectural style, but some are more suitable than others. Choosing the style before performing this type of analysis is a classic case of putting the cart before the horse.

Sam: Can't we just be super-agile, start with something tiny, and then keep iterating on it until we have the entire system?

Mara: The iterative approach you talk about doesn't quite work like that for architectural characteristics analysis. For example, it's difficult to make a system highly scalable if it wasn't designed for that.

Alex: That makes sense. I guess I need to roll up my sleeves and analyze some architectural characteristics—thanks!

What are architectural characteristics?

You have a problem. You decide, "I'm going to write some software to solve this problem!" The *thing* you're writing software about is called the ***domain***, and designing for it will occupy much of your effort—that is, after all, why you're writing software. However, it's not the only thing an architect must consider—they must also analyze ***architectural characteristics***. Here are a few examples of architectural characteristics that show how different domains have different, but often overlapping, architectural characteristics.

These make up one of the dimensions that help describe your architecture.

Notice how many of them end with "–ility".

Auditability
Banks must provide a way to verify transactions.

Data integrity
Financial transactions must be consistent and accurate.



Bank

Security
Banks require stringent security to protect financial concerns.

Both domains have scalability as an architectural characteristic they must support.

Scalability

Banks must support large numbers of concurrent users.

Online auctions must support a large number of bidders.

Usability
Auction sites must be easy to use for quick and efficient entry of bids.



Online auction

Consistency
Bids must be captured consistently and in order for auctions to work.

Reliability
Auction sites must be reliable—users don't like it if their connection drops in the middle of an auction.